Exponential-Range Mass Production of Boolean Functions

Local recovery, disjoint scheduling, and bounded congestion

Samuel Schlesinger

September 2026

Abstract

We study the cost of evaluating one Boolean function on many independent inputs. For $f : \{0,1\}^n \to \{0,1\}$, let $f^{\times t}$ evaluate f on t such inputs. Uhlig showed that $t = 2^{o(n/\log n)}$ copies preserve the sharp one-copy asymptotic. We retain the optimal order of growth throughout every fixed exponential range below the 2^n-copy scale: for every fixed $0 \le \gamma < 1$,

$$\mathrm{C}(f^{\times t}) = O_\gamma(2^n/n) \qquad \text{for all } t \le 2^{\gamma n}.$$

The bound is uniform in f and t. Using high-rate lifted evaluation codes, we sharpen it to $(1/(1-\gamma) + o_\gamma(1))2^n/n$.

We encode restrictions of f as shorter resource functions with many affine-line recovery sets. A deterministic nonuniform scheduler chooses disjoint sets, allowing each resource function to be evaluated only once. Its size is $\widetilde{O}_\ell(tq)$, where q is the field size and ℓ is the fixed geometric dimension. The scheduler tests fixed menus of candidate directions and serves half the remaining requests in each phase. The menus work for every input batch; their existence is proved, but no efficient uniform construction is given. All gates for schedule selection, resource evaluation, routing, and decoding are counted. We also retain an explicit greedy scheduler and its recursive proof of the order-of-growth bound. The Lean companion formally verifies both upper-bound variants, including the sharper coefficient.

1 Introduction

How much circuit size can be saved when the same function must be evaluated on many unrelated inputs? This is a direct-sum question in which the outputs are independent but the circuit implementing them need not be: intermediate computations may be reorganized and shared across copies.

For a Boolean function $f : \{0,1\}^n \to \{0,1\}$ and an integer $t \ge 1$, define

$$f^{\times t}(x^{(1)}, \dots, x^{(t)}) = (f(x^{(1)}), \dots, f(x^{(t)})).$$

The naive construction uses t disjoint copies of a circuit for f and gives

$$\mathrm{C}(f^{\times t}) \le t\mathrm{C}(f).$$

The mass-production problem asks how large t can become before the cost must rise above the worst-case scale for one copy. For unrestricted circuits, the naive direct-sum bound can be far from

optimal. Uhlig's theorem allows $t = 2^{o(n/\log n)}$ independent evaluations with no asymptotic increase in the leading worst-case one-copy synthesis cost [Uhl92, Weg87, Kre23]. In our basis and cost convention, the sharp one-copy coefficient is 1 [FM05]; its numerical value can depend on the basis and convention. Uhlig's 1974 paper [Uhl74] is a precursor, and Wegener's Theorem 10.2.3 gives a book account of the later result.

We prove, by a different construction, that the order-of-growth plateau reaches $2^{\gamma n}$ simultaneous evaluations for every fixed $\gamma < 1$. The asymptotic coefficient is at most $1/(1-\gamma)$ in our basis. This does not preserve Uhlig's coefficient 1 for positive fixed γ.

1.1 Main result

We fix the standard basis $\{\wedge, \vee, \neg\}$, with fan-in two for the binary gates. Constant sources, fan-out, and output designations are free; circuit size counts the $\wedge$, $\vee$, and $\neg$ gates. Any other fixed finite complete basis changes the result by at most a constant-factor simulation.

Theorem 1.1 (Exponential-range mass production)**.** *For every fixed $0 \le \gamma < 1$, there is a constant A_γ such that, for every integer $n \ge 1$, every Boolean function $f : \{0,1\}^n \to \{0,1\}$, and every integer $1 \le t \le 2^{\gamma n}$,*

$$\mathrm{C}(f^{\times t}) \le A_\gamma \frac{2^n}{n}.$$

Theorem 1.2 (Quantitative leading coefficient)**.** *For every fixed $0 \le \gamma < 1$, uniformly over Boolean functions on n bits and integers $1 \le t \le 2^{\gamma n}$,*

$$\mathrm{C}(f^{\times t}) \le \left(\frac{1}{1-\gamma} + o_\gamma(1)\right) \frac{2^n}{n}.$$

Equivalently, for every $\varepsilon > 0$, the coefficient $1/(1-\gamma) + \varepsilon$ suffices for all sufficiently large n.

The order of the quantifiers matters. Once γ is fixed, one constant A_γ works for every function and every batch size in the stated range; the bound has no factor depending on t. At the largest allowed batch size, the amortized size per requested value is therefore

$$\frac{\mathrm{C}(f^{\times t})}{t} = O_\gamma\left(\frac{2^{(1-\gamma)n}}{n}\right).$$

The quantitative bound also depends on γ and does not assert a uniform plateau as $\gamma \to 1$. This is a worst-case synthesis guarantee. For a particular function with a small circuit, the replication bound $t\mathrm{C}(f)$ may be smaller; one may always take the better of the two bounds.

There must nevertheless be a transition before the 2^n-copy scale. If $n \ge 2$ and f depends on all n variables, every circuit for $f^{\times t}$ has size at least $t(n-1)$. Hence a uniform $O(2^n/n)$ plateau cannot persist once $t = \omega(2^n/n^2)$; see Section 11. Our theorem reaches every fixed exponential rate, but leaves this near-endpoint regime open.

1.2 How the construction works

Split an input as $x = (a, z)$, with a p-bit prefix a and a d-bit suffix z, where $p + d = n$. The prefix selects one of the 2^p shorter functions $f_a(z) = f(a, z)$. The desired saving would come from using

only one circuit for each shorter function. But two inputs with the same prefix may require that function at two different suffixes, and one circuit has only one set of input wires.

Coding supplies alternative ways to answer each request. We replace the restrictions f_a by a larger bank of shorter functions, called *resources*. Each desired value can be recovered from many different sets of resources. A scheduler chooses one set for each request so that no resource is needed twice. Each resource circuit can then receive the one suffix assigned to it, and all requested answers can be reconstructed.

Uhlig serves two requests using opposite ends of an XOR chain, then scales through recursion [Uhl92, Weg87]. We replace that chain with a code offering many recovery choices, and schedule disjoint choices for an entire batch. The key is making the scheduling cheap enough to preserve the savings. Section 3 develops this comparison through a worked example.

The construction has four components.

1. **Pack restrictions.** For every fixed suffix z, encode the values $f(a, z)$ over all prefixes a in a code over $\mathbb{F}_q$. As z varies, each bit of each code coordinate defines a resource function on d variables.
2. **Recover locally.** The code coordinates are points in $\mathbb{F}_q^\ell$. A requested symbol is the sum of the $q-1$ other symbols on any line through its information point. About $q^{\ell-1}$ directions are available, giving many recovery options.
3. **Schedule disjoint recovery.** Test a fixed menu of direction assignments and choose one that serves at least half the pending requests without collisions. Reserve those recovery sets and repeat on the remaining requests. The total gate cost is $\widetilde{O}_\ell(tq)$, including selection of a schedule for the live inputs.
4. **Evaluate the resources.** Each code coordinate is now queried on at most one suffix. Apply ordinary one-copy synthesis to each resource function on $d = |z|$ variables, route its value to the requesting line, and decode.

The encoding in the first step defines the resource truth tables when the circuit is built. It is not evaluated afresh on the input wires. At evaluation time, the circuit selects recovery sets, routes suffixes, evaluates the fixed resource circuits, and decodes. The menu is also fixed in advance, but the choice of an entry depends on the input prefixes. Nonuniformity permits choosing a menu that works for all inputs; it does not permit hardwiring a single input's schedule.

1.3 Where the leading coefficient comes from

The main cost is the product of the number of resource functions and the cost of synthesizing one of them. A code whose rate tends to one, together with packing that leaves only a vanishing fraction of its capacity unused, gives

$$\underbrace{(1+o(1))2^p}_{\text{Boolean resources}} \cdot \underbrace{(1+o(1))\frac{2^d}{d}}_{\text{gates per resource}} = (1+o(1))\frac{2^n}{d}.$$

Leaving more suffix bits makes this product smaller, but leaves fewer prefix bits to provide distinct recovery options. The scheduler can accommodate $t \le 2^{\gamma n}$ while leaving $d = \delta n + O(1)$ suffix bits

for any fixed $0 < \delta < 1 - \gamma$, by taking the geometric dimension ℓ sufficiently large and then fixed. Its scheduling and routing costs are exponentially smaller than $2^n/n$. The displayed product therefore gives the coefficient $1/\delta$. Choosing δ close enough to $1 - \gamma$ gives $1/(1 - \gamma) + \varepsilon$ for any fixed $\varepsilon > 0$. For example, the bound for $t \le 2^{n/2}$ has coefficient $2 + o(1)$.

Two proof routes. The direct proof uses the menu scheduler to ensure that every resource is evaluated only once. An explicit greedy scheduler is simpler to construct but has quadratic dependence on the batch size. To control that cost, the second proof schedules smaller groups separately and recursively handles the resulting reuse of resources. Both routes prove Theorem 1.1; the direct route also gives Theorem 1.2.

For a first reading, start with the two-copy example in Section 3, then use the local-gadget statement (Proposition 4.2), the menu scheduler (Theorem 5.5), and the scheduler-sensitive composition bound (Proposition 6.1) to reach the direct proof in Section 7. Section 8 improves its coefficient. Section 8.1 explains the exact finite accounting, and Section 9 contains the alternative recursive proof. Gate-level details are collected in Section A.

1.4 Relation to earlier work

Uhlig preserves the one-copy leading term for $\log t = o(n/\log n)$ [Uhl92, Weg87]. Global truth-table lookup also handles exponentially many requests: Paul's bound is $O(n2^n)$ in our range [Pau76], while hardwiring the database in Holmgren and Rothblum's multiselection theorem gives $O(2^n)$ [HR24]. Our bound attains the smaller worst-case one-copy order $O_\gamma(2^n/n)$ for every fixed exponential rate below one.

Evaluation codes, affine-line recovery, and disjoint recovery sets are established batch-code tools [IKOS04, PV19]; high-rate lifted codes are also prior work [GKS13, HPPV20]. The contribution here is the gate-counted schedule selection and composition that turn these ingredients into the stated circuit bound. In particular, a schedule must be selected from live input addresses, and a generic polynomial-size decoder need not be small enough. Section 10 gives the detailed comparison and attribution.

Machine-checked companion. An accompanying Lean 4 development formally verifies both upper-bound variants. The explicit greedy scheduler, routing, composition bound, and equal-block induction prove a rational, discrete-exponent form of Theorem 1.1. The nonuniform development proves the universal menus, complete scheduler, high-rate code and packing, resource evaluation, and recovery in the original request order. Its parameter estimates establish Theorem 1.2, including the stated real-rate and additive-error quantifiers. Rational approximation and exponent rounding are machine checked. At revision `8dd82c9`, the Lean 4.33.1/mathlib 4.33.1 module `Algebraic.MassProduction` exposes `BlockInduction.exponentialMassProduction` for the explicit proof and `Nonuniform.realSharpMassProduction` for the nonuniform proof. The full library and regression build, tests, and lint pass; both endpoints use only Lean's standard axioms, with no proof placeholders. A small executable Python model of the menu verifier and the high-rate subcode, with exhaustive checks on tiny parameters, accompanies the manuscript source at revision `1c880d9`.

Development methodology. Generative-AI systems materially assisted mathematical experiments, literature discovery, and proof development. The required detailed disclosure appears at the end of the paper.

2 Circuit conventions and worst-case notation

Our circuits are finite directed acyclic graphs. Source vertices are input bits or the constants zero and one. Internal vertices are labelled by $\wedge$, $\vee$, or $\neg$, with the usual fan-in. A circuit for a map $F : \{0,1\}^N \to \{0,1\}^m$ has m designated output wires, which may be input wires or gate outputs and may repeat. Fan-out, constants, and output designations carry no cost; size is the number of internal gates. We write $\mathrm{C}(F)$ for the minimum size of such a circuit. All leading-constant comparisons in this paper refer to this fixed model.

Let

$$L(n) = \max_{f:\{0,1\}^n \to \{0,1\}} \mathrm{C}(f).$$

The Shannon counting lower bound and Lupanov's synthesis upper bound imply

$$L(n) = \Theta(2^n/n)$$

for every fixed finite complete basis; see [Sha49, Lup58, FM05, Weg87]. We use only the upper bound.

For $r \geq 1$, define

$$L_r(n) = \max_{f:\{0,1\}^n \to \{0,1\}} \mathrm{C}(f^{\times r}).$$

For every $r \geq 1$, one has $L_r(n) \geq L(n)$: fix all but one of the r input blocks and retain the corresponding output. Thus $2^n/n$ is the optimal worst-case order even when several outputs are requested.

For each f, choose a circuit for $f^{\times r}$ of size at most $L_r(n)$. If a construction supplies fewer than r live inputs, the unused inputs of this circuit may be fixed arbitrarily and their outputs ignored. Thus a bound for $L_r(n)$ also applies to every smaller number of copies.

All circuits below are nonuniform. Field representations, irreducible polynomials, sorting networks, and other finite combinatorial objects may be fixed as part of the circuit for each input length.

The main parameters have the following roles. Code length and dimension are measured in field symbols; multiplying by b converts either count to bits.

Parameter	Meaning
$n = p + d$, t	Input length, prefix/suffix split, and number of requests
$q = 2^b$, ℓ	Field size, bits per symbol, and fixed geometric dimension
$N = q^\ell$, K	Code length and dimension; K/N is the code rate
$D_q = (q^\ell - 1)/(q-1)$	Number of line directions through each target
C, $g = \lceil t/C \rceil$	Number of request groups and maximum group size; the direct proof takes $C = 1$

We use $\widetilde{O}_\ell(X)$ for

$$X \cdot \mathrm{poly}_\ell(n, \log q, \log(t+2), \log(C+2)).$$

Only polynomial factors are suppressed; in the parameter regimes used below, they never affect an exponential rate.

For a positive integer k, write $[k]_0 = \{0, \dots, k-1\}$.

We repeatedly use two standard circuit primitives. First, after representing $\mathbb{F}_{2^b}$ in a polynomial basis, field addition, multiplication, inversion of a nonzero element, and comparison of canonical b-bit representations have circuits of size $\mathrm{poly}(b)$. For example, inversion can be implemented by binary exponentiation to the power $2^b - 2$. Second, R records of width w can be sorted by a fixed sorting network using $O(R \log^2 R)$ compare–exchange units, each of size $O(w)$; see [Bat68]. Sorting by a leading type bit, with original positions used as tie breakers when needed, also gives stable fixed-wire compaction within the same asymptotic bound.

3 The two-copy construction as disjoint recovery

Uhlig's two-copy argument already contains the combinatorial invariant used later. We will not use its numerical bound. What matters is that each requested value has more than one representation, allowing the two requests to choose representations that do not reuse a resource at the same time.

Fix a prefix length p, put $M = 2^p$, and define the restrictions

$$f_i(z) = f(i, z), \qquad i \in \{0, \dots, M-1\}.$$

Uhlig's two-copy construction introduces the following functions; see the proof of Wegener's Theorem 10.2.3 [Weg87] and the overview in [Kre23, Section 1.2]:

$$g_0 = f_0, \qquad g_i = f_{i-1} \oplus f_i \quad (1 \le i < M), \qquad g_M = f_{M-1}.$$

Every restriction has two representations:

$$f_i = \bigoplus_{j=0}^{i} g_j = \bigoplus_{j=i+1}^{M} g_j.$$

Given two inputs $(i, z^{(1)})$ and $(j, z^{(2)})$ with $i \le j$, recover $f_i(z^{(1)})$ from the prefix set $\{0, \dots, i\}$ and $f_j(z^{(2)})$ from the suffix set $\{j+1, \dots, M\}$. The sets are disjoint, so each resource function g_j is evaluated on at most one suffix.

For example, take $p = 2$, so there are four restrictions and five resources. The requests with prefixes 1 and 2 can be answered as

$$f_1(z^{(1)}) = g_0(z^{(1)}) \oplus g_1(z^{(1)}), \qquad f_2(z^{(2)}) = g_3(z^{(2)}) \oplus g_4(z^{(2)}).$$

Resources g_0, g_1 receive $z^{(1)}$, resources g_3, g_4 receive $z^{(2)}$, and g_2 is unused. The suffixes can be completely unrelated. Even if the two prefixes coincide, the prefix and suffix recovery sets remain disjoint.

For two requests arriving in the opposite order, a comparator first orders their prefixes, conditional swaps route the two suffixes to the prefix and suffix recovery branches, and a final swap restores the output order. This costs only $\mathrm{poly}(n)$ gates.

The map $(f_0, \dots, f_{M-1}) \mapsto (g_0, \dots, g_M)$ is, up to an invertible change of information coordinates, the length-$(M+1)$ single-parity-check code. The two representations above are disjoint recovery

sets for each information coordinate. This suggests replacing the one-dimensional telescoping code by a constant-rate code with many geometric recovery sets.

The full construction keeps the same principle: synthesize arbitrary shorter resource functions, then control how often each must be evaluated. The chain offers only two recovery choices per restriction. We next replace it by a geometry with many choices, enough to serve exponentially many requests. The direct proof makes all their recovery sets disjoint; the recursive alternative allows bounded reuse across groups.

4 The local-recovery gadget

We need a code with three features: it stores the 2^p prefix values using only $O(2^p)$ bits, it offers many recovery sets for every value, and the circuit can compute the addresses in those sets cheaply. The following product code supplies all three. Proposition 4.2 is the interface used by the rest of the proof; the polynomial construction establishes that interface.

Fix an integer $\ell \geq 2$ and a field $\mathbb{F}_q$ of characteristic two, where $q = 2^b$. Set

$$s_0 = \left\lfloor \frac{q-1}{\ell} \right\rfloor$$

and fix a polynomial-basis representation of $\mathbb{F}_q$. If α_i denotes the field element whose canonical b-bit representation is the integer i, choose the efficiently indexed set

$$A = \{\alpha_0, \ldots, \alpha_{s_0-1}\}.$$

Let $\mathcal{V}$ be the space of polynomials

$$P(X_1, \ldots, X_\ell) \in \mathbb{F}_q[X_1, \ldots, X_\ell]$$

having degree less than s_0 in every variable. Repeated univariate interpolation shows that evaluation on A^ℓ uniquely determines P, and

$$\deg P \leq \ell(s_0 - 1) \leq q - 2.$$

The evaluation map

$$\mathrm{Enc} : \mathbb{F}_q^{A^\ell} \longrightarrow \mathbb{F}_q^{\mathbb{F}_q^\ell}, \qquad \big(P(u)\big)_{u \in A^\ell} \longmapsto \big(P(y)\big)_{y \in \mathbb{F}_q^\ell},$$

is a systematic linear code: the original information symbols appear unchanged at the coordinates in A^ℓ. Its length and dimension are

$$N = q^\ell, \qquad K = s_0^\ell = \Theta_\ell(q^\ell).$$

4.1 Packing Boolean restrictions

Split an n-bit input as

$$x = (a, z), \qquad |a| = p, \qquad |z| = d = n - p.$$

For each fixed suffix z, pack the 2^p bits

$$\big(f(a,z)\big)_{a\in\{0,1\}^p}$$

into K field symbols, using the fixed $\mathbb{F}_2$-basis of $\mathbb{F}_q$ and padding unused bit positions with zeros. More explicitly, interpret a prefix a as an integer $I(a) \in [2^p]_0$, and set

$$s(a) = \left\lfloor \frac{I(a)}{b} \right\rfloor, \qquad j(a) = I(a) \bmod b.$$

Write $s(a)$ in base s_0 using exactly ℓ digits $i_1(a), \ldots, i_\ell(a) \in [s_0]_0$, and define

$$u(a) = (\alpha_{i_1(a)}, \ldots, \alpha_{i_\ell(a)}) \in A^\ell.$$

Thus bit $j(a)$ of the information symbol at $u(a)$ is $f(a,z)$. Fixed-divisor division, remainder, and base conversion have Boolean circuits of size $\mathrm{poly}_\ell(p, \log q)$, so computing $(u(a), j(a))$ for all requests costs $\widetilde{O}_\ell(t)$ gates.

Choose q, among powers of two, minimally so that

$$Kb \geq 2^p.$$

For all sufficiently large p (depending on ℓ), the corresponding code dimensions at field sizes q and $q/2$ are both $\Theta_\ell(q^\ell)$. Minimality then gives

$$q^\ell b = \Theta_\ell(2^p). \tag{1}$$

Indeed, the lower bound follows from $Kb \leq q^\ell b$. For the upper bound, the preceding candidate $q/2$ has bit width $b-1$, fails the packing inequality, and has dimension $\Omega_\ell(q^\ell)$. Hence $q^\ell(b-1) = O_\ell(2^p)$; since $b \leq 2(b-1)$ for $b \geq 2$, the upper bound in (1) follows. Equivalently,

$$\ell \log_2 q + \log_2 \log_2 q = p + O_\ell(1), \qquad \log_2 q = \frac{p}{\ell} + O_\ell(\log p). \tag{2}$$

Let P_z be the polynomial whose information symbols are the packed bits. For each code coordinate $y \in \mathbb{F}_q^\ell$, define the field-valued resource

$$G_y(z) = P_z(y).$$

Each of the b coordinate bits of G_y is a Boolean function on d variables, determined by f, y, and the chosen field basis. For $r \in [b]_0$, write

$$H_{y,r}(z) = \big(G_y(z)\big)_r.$$

These definitions view the same array in two ways:

Held fixed	What varies	What the values describe
Suffix z	Code coordinate y	One codeword P_z
Code coordinate y	Suffix z	One resource function G_y

The decoder combines coordinates of a codeword. The resource circuit evaluates one column of this array at the suffix routed to it. No encoding circuit or full array is evaluated on the live inputs: the array specifies which shorter functions are synthesized.

4.2 Affine-line recovery

Lemma 4.1 (Line identity)**.** *Let $P : \mathbb{F}_q^\ell \to \mathbb{F}_q$ be represented by a polynomial of total degree at most $q-2$. For every $u \in \mathbb{F}_q^\ell$ and every nonzero direction $v \in \mathbb{F}_q^\ell$,*

$$P(u) = -\sum_{\lambda \in \mathbb{F}_q^*} P(u + \lambda v).$$

In characteristic two, the minus sign may be omitted.

Proof. The restriction $Q(\lambda) = P(u + \lambda v)$ is a univariate polynomial of degree at most $q-2$. The sum of the constant monomial over $\mathbb{F}_q$ is $q \cdot 1 = 0$ in characteristic two. For $1 \le j \le q-2$, the sum of λ^j over $\mathbb{F}_q^*$ vanishes because j is not divisible by $q-1$, and the zero term contributes nothing. Applying these identities monomial by monomial gives $\sum_{\lambda \in \mathbb{F}_q} Q(\lambda) = 0$. Separating the term $\lambda = 0$ proves the claim. □

Thus every affine line through an information point u supplies a recovery set consisting of its $q-1$ non-target points. We identify two nonzero direction vectors when one is a nonzero scalar multiple of the other and write $[v]$ for the resulting projective direction. The number of such directions through a point is

$$D_q = \frac{q^\ell - 1}{q-1} = 1 + q + \cdots + q^{\ell-1}, \qquad q^{\ell-1} \le D_q < 2q^{\ell-1}. \tag{3}$$

For fixed ℓ and $p \to \infty$, Equations (2) and (3) give the parameter dictionary

$$q^\ell = 2^{p+o(p)}, \qquad q = 2^{p/\ell + o(p)}, \qquad D_q = q^{\ell-1+o(1)} = 2^{((\ell-1)/\ell)p + o(p)}. \tag{4}$$

Two distinct lines through the same target intersect only at that target, which is absent from both recovery sets. Repeated requests for one information symbol therefore cause no additional difficulty. A later target may itself belong to the occupied set, but it blocks no direction because every new recovery set omits its own target.

The facts needed later can now be packaged without reference to the internal polynomial representation.

Proposition 4.2 (Local-recovery gadget)**.** *Fix $\ell \ge 2$ and a split $n = p + d$, with p sufficiently large depending on ℓ, and choose $q = 2^b$ by the packing rule above. For every $f : \{0,1\}^n \to \{0,1\}$ there are $q^\ell b$ Boolean resource functions $H_{y,r} : \{0,1\}^d \to \{0,1\}$ with the following properties.*

(i) *The resource count satisfies $q^\ell b = \Theta_\ell(2^p)$.*

(ii) *A prefix a determines an information point $u(a) \in A^\ell$ and a bit position $j(a) \in [b]_0$ by a circuit of size $\mathrm{poly}_\ell(p, \log q)$.*

(iii) *For every projective direction $[v]$, the set*

$$R(u(a), [v]) = \{u(a) + \lambda v : \lambda \in \mathbb{F}_q^*\}$$

has $q-1$ coordinates and recovers the requested bit by

$$f(a, z) = \bigoplus_{y \in R(u(a),[v])} H_{y,j(a)}(z).$$

There are $D_q = (q^\ell - 1)/(q-1)$ such recovery sets.

(iv) *If $U \subseteq \mathbb{F}_q^\ell$ is already occupied, at most $|U \setminus \{u(a)\}|$ directions have $R(u(a), [v]) \cap U \neq \varnothing$.*

Proof. The packing construction gives (i) and (ii). The line identity, followed by selection of coordinate bit $j(a)$ in the fixed $\mathbb{F}_2$ basis, gives (iii). For (iv), every $y \in U \setminus \{u(a)\}$ lies on exactly one projective line through $u(a)$, while the omitted target $u(a)$ blocks none. □

Thus one desired bit has become $q-1$ requests to shorter resource functions, but those coordinates may be chosen from any of D_q recovery sets. The next section turns the blocking guarantee in (iv) into a deterministic circuit and then routes only the coordinates actually selected.

5 Deterministic scheduling and routing

The scheduler sees target coordinates and chooses recovery sets; it does not evaluate any resource function. We first give a greedy implementation as a baseline, then replace its quadratic scan by fixed menus. Routing finally connects the chosen coordinates to the resource circuits. The direct proof uses one group containing all requests; grouping is needed only for the recursive alternative.

5.1 Disjoint scheduling inside one group

The geometric count says that a suitable line exists, but a circuit must find one from the target points. We do this greedily. Previously occupied points rule out directions; projective ranking and sorting let an ordinary Boolean circuit choose the first direction that remains.

Consider a multiset of g requested information points $u_1, \dots, u_g \in A^\ell$. Process the requests in order. Suppose disjoint recovery sets have been selected for requests $1, \dots, j-1$, and let U be their union. Then

$$|U| = (j-1)(q-1) < gq.$$

For the target u_j, every point $y \in U \setminus \{u_j\}$ forbids at most one projective direction, namely the direction of $y - u_j$. The point u_j itself forbids none because it is omitted from the new recovery set.

Lemma 5.1 (Projective indexing)**.** *Projective directions in $\mathbb{F}_q^\ell$ have rank and unrank circuits of size* $\mathrm{poly}_\ell(b)$*, with ranks in $[D_q]_0$.*

Implementation idea. The indexing normalizes the first nonzero coordinate to one and orders the normalized vectors by the position of that coordinate and then by their remaining base-q digits. The explicit circuits are given in Section A.1.

Lemma 5.2 (Greedy line scheduler)**.** *If*

$$gq < D_q, \tag{5}$$

then every multiset of g target points admits pairwise disjoint affine-line recovery sets. Given the targets, the recovery sets can be selected by a Boolean circuit of size

$$\widetilde{O}_\ell(g^2 q).$$

Proof idea. At stage j, the blocking property in Proposition 4.2 leaves a valid direction because fewer than D_q directions are forbidden. The circuit ranks the forbidden directions, sorts those ranks, and selects the least missing one; stage j costs $\widetilde{O}_\ell(jq)$. The treatment of duplicate targets, sentinel ranks, and the least-missing-rank circuit is given in Section A.2. Summing over the stages gives the stated cost.

5.2 A nonuniform scheduler with linear dependence on the batch size

The quadratic scan can be replaced by phases that accept half the remaining requests. We prove that a small fixed menu of candidate directions works for every possible phase state, then give a circuit that tests the menu. The existence proof uses randomness only to choose constants of the circuit; its evaluation is deterministic on every input.

There are three steps. First, one random direction assignment succeeds with very high probability for any fixed state. Second, a union bound supplies a short menu that contains a successful assignment for every state. Third, a sorting circuit finds such an assignment without multiplying the occupied-set cost by the menu length. Here g is the original group size, k is the number of pending requests, and U contains the points reserved earlier. A menu entry is a tuple of k directions, one for each pending request.

Lemma 5.3 (A partial schedule with exponentially small failure)**.** *Put $N = q^\ell$. Suppose $512gq \le D_q$, let $1 \le k \le g$, and let U be the union of at most g affine-line recovery sets. Fix any k targets, and choose one projective direction independently and uniformly for each target. Call a candidate recovery set* clean *if it avoids U and every other candidate recovery set. Then*

$$\Pr\{\textit{fewer than } \lceil k/2 \rceil \textit{ sets are clean}\} \le 2^{-k}.$$

Proof. For any fixed set B, a random recovery set through any fixed target meets B with probability at most $|B|/D_q$: each point other than the target blocks exactly one direction. Consequently, the probability of meeting U is at most $a_0 = g(q-1)/D_q$, and that of meeting another fixed candidate recovery set is at most $a_1 = (q-1)/D_q$.

Pairwise collision events are dependent, so we cannot multiply these bounds directly. Instead, we extract a set of bad requests whose collision witnesses all lie outside that set, and condition on the outside directions.

Form a graph on $\{0, 1, \dots, k\}$. Join a request to vertex 0 when its recovery set meets U, and join two requests when their recovery sets meet. The nonclean requests are exactly the nonisolated vertices other than 0. Take a spanning forest of the nontrivial components and two-color it. Every nonclean request has a forest neighbor of the opposite color. If at least $\lceil k/2 \rceil$ requests are nonclean, one color contains at least $h = \lceil k/4 \rceil$ of them. Choose exactly h such requests as a set S, excluding vertex 0. Every request in S then has a collision witness outside S, either vertex 0 or an unselected request.

For each fixed set S of size h, condition on all directions outside S. Their recovery sets, together with U, form a fixed set of at most $(g+k)(q-1)$ points. Each selected request meets that set with probability at most $a_0 + ka_1$. These h tests are conditionally independent, since each uses only its own unexposed direction. Averaging over the complementary directions and taking a union bound

over S gives

$$\Pr\{\text{such a collision cut exists}\} \le \binom{k}{h}(a_0 + ka_1)^h \le 2^k(1/256)^{k/4} = 2^{-k}.$$

Here $a_0 + ka_1 \le 2gq/D_q \le 1/256$. Failure of the stated progress condition implies at least $\lceil k/2 \rceil$ nonclean requests, so this proves the claim. The forest argument does not assume independence of collision events; in particular it includes many lines meeting one candidate line. □

Lemma 5.4 (Universal menus)**.** *Under the same slack condition, for each $1 \le k \le g$ there is a fixed menu of*

$$r_k = \left\lceil \frac{g(1 + 3\log_2 N) + 1}{k} \right\rceil \tag{6}$$

direction k-tuples such that every set U described in Lemma 5.3 and every ordered list of k targets has a menu entry with at least $\lceil k/2 \rceil$ clean recovery sets.

Proof. A recovery set is specified by an anchor and a direction, giving at most ND_q descriptions. Allowing an unused marker in each of g slots, the number of occupied-set descriptions is at most $(1 + ND_q)^g$. The targets have at most N^k descriptions. Since $D_q \le N$ and $k \le g$, the total number of states is at most

$$(1 + ND_q)^g N^k \le 2^{g(1+3\log_2 N)}.$$

Counting line descriptions is essential: treating U as an arbitrary list of $g(q-1)$ points would introduce an extra factor of q in this exponent. Choose r_k candidate tuples independently. For a fixed state, the probability that every tuple fails is at most 2^{-kr_k} by Lemma 5.3. A union bound over states is strictly below one by (6). Thus a menu working for all states exists. Fix one such menu as part of the circuit. Its choice depends on g, k, q, ℓ, not on the input targets or on the function being mass-produced. □

Theorem 5.5 (Nonuniform linear scheduling)**.** *If $512gq \le D_q$, a deterministic nonuniform Boolean circuit maps every ordered multiset of g targets in $\mathbb{F}_q^\ell$ to enumerated pairwise disjoint affine-line recovery sets using $\widetilde{O}_\ell(gq)$ gates. More explicitly, its size is $O(gq\Lambda^5)$ for $\Lambda = \ell \log_2 q + \log_2(g+2)$. The circuit's fixed menus have total description length $O(g(\log N)^2 \log(g+2))$ bits.*

Proof. Maintain k active targets and the occupied set from the $g - k$ accepted requests. Initially $k = g$ and U is empty. Test the menu from Lemma 5.4, choose its first successful entry, and accept exactly the first $\lceil k/2 \rceil$ clean requests in that entry. These sets avoid U and each other. Compact the other $\lfloor k/2 \rfloor$ requests, retaining their original identifiers. This gives a fixed sequence of at most $1 + \lfloor \log_2 g \rfloor$ phase sizes, and never occupies more than g recovery sets. After the last phase, sort accepted records by original identifier to enumerate the answer sets in request order.

It remains to bound the cost of testing a menu. Generate records for all $r_k k(q-1)$ candidate points, labelled by candidate, request, and line position, and append the $(g-k)(q-1)$ occupied points *once*. Since $r_k k = O(g \log N)$, there are $O(gq \log N)$ records. Use these fixed passes:

(i) Sort by point, with occupied records first at equal points. Along each equal-point run propagate an occupancy bit. In sorted position i, the update is $e_i = \text{occupied}_i \vee ((y_i = y_{i-1}) \wedge e_{i-1})$, with a dummy predecessor at the boundary. Thus every candidate point learns whether it lies in U.

(ii) Compact candidate records and sort them by (candidate, y). Mark a record when its predecessor or successor has the same key. Points on one recovery line are distinct, so such a match is exactly an intersection between two requests in the same candidate.

(iii) Sort by (candidate, request, line position) and OR the $q-1$ bad-point flags for each request. Count the clean requests in each candidate and use priority selection to choose its first successful entry. Fixed multiplexers and compaction implement the acceptance step.

Every record has polynomial width in $\ell \log q$ and $\log(g+2)$, and a constant number of sorting networks and linear scans suffices. Thus a phase has size $O(gq\Lambda^4)$: there are $O(gq\Lambda)$ records, with $O(\Lambda)$ bits per point record and $O(\Lambda^2)$ sorting-network comparators per record. Candidate directions and line scalars are hardwired, so generating a candidate point adds a constant field vector to the target using $O(\ell \log q)$ gates. Selecting and compacting whole line records costs at most $O(gq\Lambda^3)$ additional gates. There are $O(\Lambda)$ phases, proving the explicit bound. The menus contain $O(g \log N)$ directions per phase, each of $O(\log N)$ bits, giving the description bound. No search for a menu is performed on the input wires, and no N-entry occupancy table is needed. □

The theorem supplies a circuit-existence result in the paper's nonuniform model. It does not give an efficient uniform constructor for the menus. In contrast, the earlier greedy scheduler can be constructed directly once the field representation is fixed. The following grouped construction records that explicit alternative.

5.3 Request groups

Partition the t requests into C fixed groups, each of size at most

$$g = \left\lceil \frac{t}{C} \right\rceil .$$

Apply Lemma 5.2 independently inside each group. A code coordinate is used at most once per group and therefore at most C times overall. The inequalities $\lceil t/C \rceil \le t/C + 1$ and $2t \le t^2/C + C$, the latter being the arithmetic–geometric mean inequality, show that the total scheduling cost is

$$C \cdot \widetilde{O}_\ell(g^2 q) = \widetilde{O}_\ell\left(\frac{t^2 q}{C} + Cq \right). \tag{7}$$

The construction is valid whenever (5) holds for $g = \lceil t/C \rceil$, that is, whenever

$$\left\lceil \frac{t}{C} \right\rceil q < D_q. \tag{8}$$

The parameter C exposes the central tradeoff. More groups mean fewer requests per group, making disjoint scheduling easier and cheaper. But they also permit each resource coordinate to receive more suffixes, making the recursive C-copy problem harder. The recursion works by finding an exponential choice of C that satisfies both demands. In the proof below, one encoded block supplies the room inside a group and the remaining blocks supply the capacity across groups.

5.4 Scatter and gather

The scheduler produces a sparse list of resource addresses, whereas the resource circuits occupy fixed positions in the circuit. *Scatter* sends each requested suffix to its resource position; *gather* returns the resulting value to the request that needs it. Sorting implements both maps with cost proportional, up to polynomial factors, to the lists being connected.

Every non-target point of a selected line produces an incidence record containing a request identifier, a group, a line position, the point's code coordinate y, and the suffix to be evaluated. There are exactly $t(q-1) < tq$ such records. Inside one group, each coordinate appears in at most one record.

The routing consists of four fixed sorting-and-local-update passes. Here "key" means (group, y); any unlisted tie is broken by the record's original position. The record-type order is pass-specific and is shown explicitly in the table.

Pass	Records	Sort order	Local result
Scatter match	incidence, slot	$(\text{key}, \text{type})$; incidence before slot	a matched slot takes its predecessor's suffix; otherwise it takes a dummy
Scatter order	incidence, filled slot	$(\text{type}, \text{key})$; slots first	slots are in canonical (group, y) order
Gather match	value, saved incidence	$(\text{key}, \text{type})$; value before incidence	a matched incidence takes its predecessor's value
Gather order	value, updated incidence	$(\text{type}, \text{request}, \text{line position})$; incidences first	decoder inputs are in canonical request order

Scatter records carry a d-bit suffix. After resource evaluation, value records carry the b-bit field value $G_y(z)$. The gather passes are payload-agnostic for any payload whose width is polynomial in the displayed parameters.

The gather half is also an instance of multiselection. Equivalently, it is a partial-function inner join in the sense of Holmgren and Rothblum's Proposition 25 and Sections 6.1–6.2 [HR24, Proposition 25 and Sections 6.1–6.2]. Write $M = Cq^\ell$ for the number of value slots and $R = t(q-1)$ for the number of incidences. Disjointness within each group, together with the group component of the key, makes all incidence keys globally unique, so $R \le M$. Each incidence in canonical request–line-position order supplies the index of its key; applying their binary main theorem independently to the b bit positions therefore gives size $O(bM + bR\log^3(M+2))$ [HR24, Main theorem], within the bound below. We retain the four-pass construction because it handles both the sparse-to-dense scatter and the gather in one self-contained routing lemma; multiselection by itself does not provide the scatter half.

Lemma 5.6 (Record routing)**.** *Suppose each key (group, y) occurs in at most one incidence record. The suffixes can then be scattered into slots indexed by $(\text{group}, y) \in [C]_0 \times \mathbb{F}_q^\ell$, and the resulting resource values can be gathered back to their request records, using circuits of total size*

$$\widetilde{O}_\ell(tq + Cq^\ell).$$

Implementation idea. The four passes in the table use a constant number of sorting networks on $O(tq + Cq^\ell)$ records of polynomial width. The exact fixed-wire treatment of unmatched records and array boundaries is given in Section A.3.

The lemma is the reason no tq^ℓ crossbar appears. Only the actual incidences and one dummy record per resource slot are routed.

6 The composition bound

We now assemble the circuit. Its size has three parts: the bank of shorter resource circuits, the scheduler that chooses their inputs, and the routing and decoding around them. The number C of groups determines how many inputs each resource must handle. Setting $C = 1$ gives the direct proof; larger C leads to recursion.

Bounded-demand invariant. Fix f and the local-recovery gadget above. Each $H_{y,r}$ is one fixed Boolean function on d variables. Inside each request group, disjoint recovery ensures that coordinate y occurs at most once; hence the pair (y, r) receives at most one suffix from each of the C groups. Place those suffixes in the canonical group slots and pad unused slots with a fixed dummy suffix. One circuit for $H_{y,r}^{\times C}$, of size at most $L_C(d)$, serves exactly this bounded demand. There are $q^\ell b$ such pairs, and no sharing between different pairs is assumed.

The next statement isolates the circuit-theoretic information that a local recovery construction must supply. Combinatorial existence of disjoint recovery sets is not enough: their selection cost appears explicitly as S.

Proposition 6.1 (Scheduler-sensitive composition)**.** *Fix $\ell \geq 2$ and the local-recovery gadget for a split $n = p + d$. Let $C \geq 1$, set $g = \lceil t/C \rceil$, and suppose that, for every group size $1 \leq h \leq g$, a Boolean circuit of size at most $S(g, q)$ maps every multiset of h information points to enumerated pairwise disjoint affine-line recovery sets. Then every $f : \{0,1\}^n \to \{0,1\}$ satisfies*

$$\mathrm{C}(f^{\times t}) \leq \underbrace{q^\ell b\, L_C(d)}_{\text{resource circuits}} + \underbrace{CS(g,q)}_{\text{schedule selection}} + \underbrace{\widetilde{O}_\ell\Big(tq + Cq^\ell\Big)}_{\text{routing and decoding}}. \tag{9}$$

Proof. Map the requested prefixes to their information points and bit positions, and partition them into the C fixed groups. The mapping costs $\widetilde{O}_\ell(t)$ gates, and the assumed schedulers cost $CS(g, q)$. Their within-group disjointness gives the bounded-demand invariant above.

For every pair (y, r), evaluate $H_{y,r}^{\times C}$ on the canonical group slots, padding absent requests by a fixed dummy suffix. These resource circuits cost $q^\ell b L_C(d)$ in total. The scatter and gather cost $\widetilde{O}_\ell(tq + Cq^\ell)$ by Lemma 5.6. Finally, XOR the $q - 1$ returned field values for each request and select the requested coordinate bit. This costs $O(tqb)$ and is correct by Lemma 4.1. All remaining mapping and decoding costs are absorbed in the displayed overhead. The polynomials P_z and functions $H_{y,r}$ specify nonuniform resource truth tables; no run-time encoder is being assumed. □

Proposition 6.2 (Composition bound)**.** *Fix $\ell \geq 2$. Let $n = p + d$ with p sufficiently large depending on ℓ, choose $q = 2^b$ minimally so that $s_0^\ell b \geq 2^p$, and suppose Equation (8) holds. Then every $f : \{0,1\}^n \to \{0,1\}$ satisfies*

$$\mathrm{C}(f^{\times t}) \leq \underbrace{q^\ell b\, L_C(d)}_{\text{recursive resource evaluation}} + \widetilde{O}_\ell\left(\underbrace{\frac{t^2 q}{C}}_{\text{line scheduling}} + \underbrace{tq}_{\text{incidences and decoding}} + \underbrace{Cq^\ell}_{\text{resource slots}} \right). \tag{10}$$

The first term is recursive: there are $q^\ell b$ Boolean resource functions, each evaluated on at most C suffixes of length d. The other three terms account for scheduling, the selected line incidences, and the padded resource slots. Increasing C makes the groups smaller, but gives every resource more suffixes to process. The induction chooses C so that the recursive term stays at scale $2^n/n$ and all three bookkeeping terms are exponentially smaller.

Proof. Put $g = \lceil t/C \rceil$. The scheduling condition lets us apply Lemma 5.2 with $S(g, q) = \widetilde{O}_\ell(g^2 q)$. Moreover,

$$Cg^2 q \le C\left(\frac{t}{C} + 1\right)^2 q = O\left(\frac{t^2 q}{C} + tq + Cq\right).$$

Apply Proposition 6.1; the tq term is already present, and Cq is absorbed by Cq^ℓ because $q \ge 2$ and $\ell \ge 2$. □

7 A direct proof without recursion

The parameter choice balances storage against recovery capacity. Keeping $d \approx \delta n$ suffix bits leaves about $2^{(1-\delta)n}$ encoded prefix bits. Since a recovery set occupies about q points while a target has about $q^{\ell-1}$ directions, the scheduler needs t to be smaller than a constant multiple of $q^{\ell-2}$. The inequality below expresses exactly this requirement at the level of exponential rates.

Direct proof of Theorem 1.1. Fix $\gamma < 1$, choose any fixed $0 < \delta < 1 - \gamma$, and put $d = \lceil \delta n \rceil$ and $p = n - d$. Choose a fixed $\ell \ge 2$ large enough that

$$\gamma < (1 - 2/\ell)(1 - \delta). \tag{11}$$

Use the product-code gadget with its minimal packing field. Its parameters satisfy

$$\log_2 q = (1 - \delta)n/\ell + o(n), \qquad \log_2 D_q = (1 - 1/\ell)(1 - \delta)n + o(n).$$

Thus (11) implies $512tq \le D_q$ for all sufficiently large n and every $t \le 2^{\gamma n}$. Apply Theorem 5.5 and Proposition 6.1 with $C = 1$. This gives

$$\mathrm{C}(f^{\times t}) \le q^\ell b L(d) + \widetilde{O}_\ell(tq + q^\ell).$$

The first term is $O_\ell(2^p 2^d/d) = O_\gamma(2^n/n)$. The two overhead exponents are at most $\gamma + (1 - \delta)/\ell < 1$ and $1 - \delta < 1$, respectively, with fixed positive margins. All suppressed polynomial factors are therefore negligible compared with $2^n/n$. Enlarge the constant to cover the finitely many initial lengths by ordinary circuit replication. □

8 High-rate codes and the leading coefficient

The preceding proof has two constant losses. First, the product code's rate approaches $\ell^{-\ell}$: it stores substantially more symbols than the information it carries. Second, rounding up to the next allowed field size can leave a constant fraction of the information positions unused. We remove the first loss with a code of rate tending to one and the second with several smaller codewords, so that only the last one needs padding. The scheduler and decoder continue to use the same line-recovery interface.

Lifted Reed–Solomon codes already provide rate tending to one with line-parity recovery [GKS13, HPPV20]. We give an elementary sufficient monomial construction below, followed by the packing and circuit-cost accounting.

Lemma 8.1 (High-rate line-parity code)**.** *For integers $\ell \geq 2$ and $h \geq \lceil \log_2 \ell \rceil$, put $q = 2^b$ and $N = q^\ell$. There is a linear code on $\mathbb{F}_q^\ell$ of dimension K such that*

$$\frac{K}{N} \geq 1 - (1 - 2^{-\ell h})^{\lfloor b/h \rfloor} = 1 - o_\ell(1) \qquad (b \to \infty), \tag{12}$$

and each word P satisfies

$$P(u) = \sum_{\lambda \in \mathbb{F}_q^*} P(u + \lambda v)$$

for every target u and every nonzero direction v. The code can be made systematic on a fixed set of K evaluation points.

Proof. The low-degree condition in Lemma 4.1 was sufficient for line parity, but stronger than necessary. For the sum over $\mathbb{F}_q$ to vanish, it is enough that every positive degree appearing on a line avoid multiples of $q - 1$. We enforce this weaker condition using a common block of zero binary digits.

Consider reduced monomials $X_1^{e_1} \cdots X_\ell^{e_\ell}$ with $0 \leq e_i < 2^b$. Partition the first $h\lfloor b/h \rfloor$ binary digit positions into disjoint blocks of length h. Retain a monomial if some block is zero in *all* of its exponents. The fraction retained is exactly $1 - (1 - 2^{-\ell h})^{\lfloor b/h \rfloor}$. These monomials are linearly independent as evaluation functions, by repeated univariate interpolation.

To check line parity, expand a retained monomial at $u + \lambda v$. In characteristic two, every possibly nonzero term in this expansion has λ-degree $J = \sum_i j_i$, where the one-bits of j_i are a subset of those of e_i. This follows directly by writing $(u_i + \lambda v_i)^{e_i}$ as a product over the one-bits of e_i and using $(a + c)^{2^r} = a^{2^r} + c^{2^r}$.

Suppose the common zero block starts at position s. Every j_i has the same zero block, so its residue modulo 2^{s+h} is at most $2^s - 1$. Consequently,

$$J \bmod 2^{s+h} = \sum_i (j_i \bmod 2^{s+h}) \leq \ell(2^s - 1) \leq 2^{s+h} - \ell.$$

There is no wraparound in this equality. If J were a positive multiple $a(2^b - 1)$, we would have $1 \leq a \leq \ell - 1$: the zero block makes every $j_i < 2^b - 1$. Since $s + h \leq b$ and $a < 2^{s+h}$, its residue would instead be $2^{s+h} - a \geq 2^{s+h} - \ell + 1$, a contradiction. Thus every positive λ-degree is not divisible by $q - 1$. The monomial-sum calculation in Lemma 4.1 gives zero for the sum over $\mathbb{F}_q$, including the constant term. Linearity proves line parity for the span of the retained monomials.

Finally, choose K independent coordinate columns of a generator matrix. Evaluation on those coordinates is an isomorphism to $\mathbb{F}_q^K$, giving a systematic information set. The choice and any change of basis are offline parts of the nonuniform code specification. □

The information set need not have the product code's convenient indexing. This does not force a separate lookup circuit of size K for each request. Store its ordered list of K points as a hardwired table, and sort its table records together with all t live symbol indices, placing a table record before its queries at an equal index. Propagate the table value along each equal-index run, then sort queries back to request order. This costs $\widetilde{O}_\ell(K + t)$ gates and handles repeated indices. It is the same batched lookup primitive used for occupancy broadcast, with a point as payload.

Proof of Theorem 1.2. Fix $0 < \delta < 1 - \gamma$, put $d = \lceil \delta n \rceil$, $p = n - d$, and choose a fixed ℓ satisfying (11). Instead of the minimal packing field, take

$$b = \left\lfloor \frac{p - 3\log_2(n+2)}{\ell} \right\rfloor, \qquad q = 2^b, \qquad B = \left\lceil \frac{2^p}{Kb} \right\rceil,$$

for sufficiently large n, where K is the dimension in Lemma 8.1. Use B separate copies of this code. Each has Kb information bits, so they hold the 2^p prefix values with only the last code's information positions padded. Because $K/N \to 1$ and $Nb = o(2^p)$, their total number of Boolean resources is

$$2^p \le BNb \le \frac{N}{K}\, 2^p + Nb = (1 + o(1))2^p. \tag{13}$$

Here $B = \mathrm{poly}_\ell(n)$ and $\log_2 q = (1-\delta)n/\ell + O_\ell(\log n)$.

Fixed-divisor arithmetic maps each prefix to its code number, information index, and bit position. The batched table lookup above supplies all target points. Schedule these t targets together, even when they refer to different code copies. By Theorem 5.5, the resulting recovery sets are globally disjoint. Attaching the requested code number to each incidence therefore gives unique keys (code, y). Route the $t(q-1)$ incidences against BN slots by the four-pass construction of Lemma 5.6. In particular, neither the scheduler nor the incidence list is replicated once per code copy.

For each of the BNb Boolean resource functions on d variables, use the sharp one-copy synthesis bound $L(d) \le (1 + o(1))2^d/d$ in the stated basis [FM05]. The complete bound is

$$\mathrm{C}(f^{\times t}) \le BNbL(d) + \widetilde{O}_\ell(tq + BN + K) \le \left(\frac{1}{\delta} + o(1)\right) \frac{2^n}{n}.$$

Indeed, (13) controls the resource term; the other terms have exponents at most $\gamma + (1-\delta)/\ell < 1$ and $1 - \delta < 1$. All estimates are uniform over f and the allowed t.

For every fixed $\delta < 1 - \gamma$ we have proved a limiting upper coefficient $1/\delta$. Taking the infimum of these bounds as δ increases to $1 - \gamma$ proves the theorem. This last step takes a limit of fixed-parameter bounds; it makes no assertion uniform in growing ℓ or in $\gamma \to 1$. □

8.1 Exact accounting in the formal proof

The formalization makes several invariants in the preceding argument explicit. They explain how the complete circuit preserves the leading coefficient, including at input lengths that do not fit a field block. This subsection is independent of the direct asymptotic proof above: it provides a finite accounting version for readers comparing the paper with the Lean theorem.

Request identity and inactive slots. Each request carries a distinct fixed identifier, even when its target and suffix coincide with another request's. The scheduler maintains a permutation of accepted and pending requests, together with disjoint recovery lines for the accepted prefix. Each phase preserves all original payloads. For regular array sizes, a line uses q scalar slots: the zero scalar is marked inactive, and its recovered Boolean contribution is fixed to zero. After XOR recovery, only the identifier and the one-bit answer need to be sorted back into request order. Thus restoring output order does not require another permutation of the full point lists.

Charge exactly the resource bank. A single hardwired table indexed by the source prefix can return the code number, information point, and basis-bit selector together. Batched lookup handles repeated prefixes and removes runtime division from this version of the construction. The resource key is (code number, point, basis bit); valid scheduled incidences have distinct keys. Evaluate precisely the $R = BNb$ Boolean resource circuits, once each. Padding the sorting networks adds inactive routing records, not resource evaluations. This distinction avoids a constant-factor loss in the main term. If $\widehat{L}(d)$ bounds the chosen one-copy synthesis circuits, the full cost is bounded by

$$R\widehat{L}(d) + H, \qquad H \le c_\ell 2^p n^7$$

in the parameter regime below, with c_ℓ independent of n, f, t. For each fixed positive integer Q, the actual synthesis circuits eventually admit $\widehat{L}(d) = \lfloor (Q+1)2^d/(Qd) \rfloor$. This checked polynomial envelope is coarser than the optimized pass count in Theorem 5.5; both give the required negligible overhead.

Whole field blocks, arbitrary input lengths. One exact parameter choice fixes positive integers h, ℓ, G, S, with $\ell \le 2^h$, and sets

$$P = \ell h + 1 < S, \qquad m = \lfloor n/S \rfloor, \qquad b = hm, \qquad p = Pm, \qquad d = n - p.$$

Choose $G + h + 1 \le h(\ell - 1)$ and $\gamma < G/S$. For sufficiently large m, the scheduler batch 2^{Gm} covers the requested copies, including rounding, and has $512\, 2^{Gm} q \le D_q$. Also $2^{Gm} q \le 2^p$. Put the incomplete input block into the suffix; then

$$p + d = n, \qquad \frac{d}{n} \ge \frac{S - P}{S} > 0.$$

Integer slopes can make P/S arbitrarily close to γ from above while keeping all these inequalities strict. All dimensions and slopes are fixed before n varies.

For $A = 2^{\ell h}$ the retained dimension on this subsequence is exactly $K = A^m - (A-1)^m$. At a fixed positive integer precision Q, the elementary rate estimate is

$$m \ge Q(A-1) \quad \Longrightarrow \quad QA^m \le (Q+1)K.$$

The formal packing uses $B = \lfloor 2^p/(Kb) \rfloor + 1$, charging at most one extra codeword. Since $Nb = 2^{\ell hm} hm$ and $p = (\ell h + 1)m$, eventually $QNb \le 2^p$. Consequently $QR \le (Q+2)2^p$. The number B need not be polynomial in this alternative parameter choice; the bounds on the total resource bank and the key widths are what matter. Finally, $m \le d$ and every fixed polynomial in n is eventually smaller than 2^m. These facts absorb H after normalization by $n/2^n$.

Uniform quantifiers. For a rational rate $u/v < 1$ and every positive integer precision J, the checked integer endpoint supplies one cutoff, uniform over f and all positive $t \le 2^{\lfloor un/v \rfloor}$, with

$$J(v-u)n\, \mathrm{C}(f^{\times t}) \le (J+1)v2^n.$$

The final real-rate theorem chooses a slightly larger rational rate and a large enough precision inside the requested additive error. Upward exponent rounding is covered by the scheduler's strict capacity margin. The result has exactly the real-rate and ε quantifiers of Theorem 1.2, without substituting growing dimensions into a fixed-dimension estimate.

9 The explicit equal-block alternative

This second proof uses the greedy scheduler and no existential menus. It proves Theorem 1.1, but does not establish Theorem 1.2. Apply the composition bound recursively. At level k, split the input into $k+1$ approximately equal blocks, encode one block, and invoke the preceding level on the remaining k.

For an integer $k \geq 1$, let P_k denote the assertion that, for every fixed $0 \leq \gamma < k/(k+1)$, there is a constant $A_{k,\gamma}$ such that, for all sufficiently large n, every $f : \{0,1\}^n \to \{0,1\}$, and every integer $1 \leq t \leq 2^{\gamma n}$,

$$\mathrm{C}(f^{\times t}) \leq A_{k,\gamma} \frac{2^n}{n}.$$

Every comparison below has a fixed positive linear margin in its base-two logarithm. We use without further comment that such a margin absorbs the $o(n)$ terms, every fixed polynomial factor hidden by $\widetilde{O}$, and the $O(1)$ changes caused by floors and ceilings.

9.1 Two equal blocks: the base case

Lemma 9.1 (Two-block base)**.** P_1 *holds.*

Here $C = 1$, so no mass-production hypothesis is used inside a resource. The two equal blocks make the resource term proportional to $2^{n/2}L(n/2)$; the one-half threshold comes from the quadratic scheduler term $t^2 q$.

Proof. Fix $0 \leq \gamma < 1/2$, let $\eta = 1/2 - \gamma > 0$, and suppose $t \leq 2^{\gamma n}$. Put

$$m = \left\lfloor \frac{n}{2} \right\rfloor, \qquad p = m, \qquad d = n - m, \qquad C = 1.$$

Thus $n = 2m + O(1)$ and all recovery sets will be globally disjoint. In block units, the request and code parameters are

$$\log_2 t \leq m - \eta n + O(1), \qquad \log_2 q = \frac{m}{\ell} + o(n), \qquad \log_2 D_q = \left(1 - \frac{1}{\ell}\right) m + o(n).$$

Choose a sufficiently large fixed ℓ so that

$$\frac{1}{\ell} < \eta. \tag{14}$$

Since $2m/\ell \leq n/\ell < \eta n$, this gives

$$m - \eta n + \frac{m}{\ell} < m - \frac{m}{\ell}.$$

The strict linear margin now implies $tq < D_q$ for all sufficiently large n.

The resource term in Equation (10) is, by Shannon–Lupanov synthesis,

$$q^\ell b L(d) = O_\ell\left(2^p \frac{2^d}{d}\right) = O_\gamma(2^n/n).$$

The scheduler exponent is

$$\log_2(t^2 q) \le 2m - 2\eta n + \frac{m}{\ell} + o(n) < n,$$

where the strict inequality follows from (14). Likewise, $\log_2(tq) \le m - \eta n + m/\ell + o(n) < n$ and $\log_2 q^\ell = m + o(n) < n$. After the suppressed polynomial factors are restored, every overhead term is $2^{(1-\Omega_\gamma(1))n}$ and is negligible compared with $2^n/n$. The same strict margins absorb all rounding. □

9.2 Adding one equal block

Lemma 9.2 (Equal-block step)**.** *For every integer $k \ge 2$, P_{k-1} implies P_k.*

Proof. Fix $0 \le \gamma < k/(k+1)$. If $\gamma < (k-1)/k$, the assertion is already P_{k-1}. We may therefore assume

$$\frac{k-1}{k} \le \gamma < \frac{k}{k+1}.$$

Let

$$\eta = \frac{k}{k+1} - \gamma > 0, \qquad m = \left\lfloor \frac{n}{k+1} \right\rfloor, \qquad p = m, \qquad d = n - m.$$

Write $n = (k+1)m + r$ with $0 \le r \le k$, and choose the number of request groups to be

$$C = \left\lceil 2^{(k-1)m - \eta n/2} \right\rceil. \tag{15}$$

This choice splits the available slack in half. Dividing the requests among C groups puts each group $\eta n/2$ below the one-block exponent m; measured on the remaining d bits, the exponent of C is also a fixed amount below the threshold supplied by P_{k-1}. The assumption $\gamma \ge (k-1)/k$ ensures that the exponent in (15) is positive for all sufficiently large n.

Let $\beta = (k-1)/k$. Since $d = km + r$,

$$\beta d - \log_2 C = \frac{\eta n}{2} + \frac{k-1}{k} r + O(1).$$

In particular, $C \le 2^{(\beta - \eta/4)d}$ for all sufficiently large n. The exponent $\beta - \eta/4$ is fixed and strictly below the threshold for P_{k-1}, so the inductive hypothesis gives

$$L_C(d) = O_{k,\gamma}(2^d/d).$$

By Equation (1), the resource term is therefore

$$q^\ell b L_C(d) = O_{k,\gamma,\ell}\left(2^m \frac{2^d}{d}\right) = O_{k,\gamma,\ell}(2^n/n).$$

The remaining estimates are easiest to read in block units. Since $\log_2 t \le \gamma n = km - \eta n + O_k(1)$, the size $g = \lceil t/C \rceil$ of a request group satisfies

$$\log_2 g \le m - \frac{\eta n}{2} + O_k(1).$$

The code parameters satisfy

$$\log_2 q = \frac{m}{\ell} + o(n), \qquad \log_2 D_q = \left(1 - \frac{1}{\ell}\right) m + o(n).$$

Choose a sufficiently large fixed ℓ so that

$$\frac{2}{(k+1)\ell} < \frac{\eta}{2}. \tag{16}$$

Then $2m/\ell < \eta n/2$, and hence

$$m - \frac{\eta n}{2} + \frac{m}{\ell} < m - \frac{m}{\ell}.$$

Together with (3), this is exactly the strict exponent margin required for Equation (8).

Finally, the complete overhead ledger in Equation (10) is

$$\begin{aligned}
\log_2\left(\frac{t^2 q}{C}\right) &\le (k+1)m - \frac{3\eta n}{2} + \frac{m}{\ell} + o(n) < n, \\
\log_2(tq) &\le n - m - \eta n + \frac{m}{\ell} + o(n) < n, \\
\log_2(Cq^\ell) &\le n - m - \frac{\eta n}{2} + o(n) < n.
\end{aligned}$$

The first strict inequality uses $(k+1)m \le n$ and (16); the other two use $m = \Theta_k(n)$ and $\ell > 1$. Thus every overhead term is exponentially smaller than $2^n/n$. This proves P_k. □

Alternative proof of Theorem 1.1. By Lemmas 9.1 and 9.2, P_k holds for every $k \ge 1$. Fix $0 \le \gamma < 1$ and choose k so that $\gamma < k/(k+1)$. Then P_k gives the asserted bound beyond a threshold N_γ. For the finitely many integers $1 \le n < N_\gamma$, use $\mathrm{C}(f^{\times t}) \le t\mathrm{C}(f)$ and enlarge A_γ to dominate $nt\mathrm{C}(f)/2^n$ over all relevant n, f, and t. This proves the statement for every $n \ge 1$. □

10 Relation to earlier work

The closest asymptotic guarantees and the main coding antecedent are summarized below. Here "one-copy leading term" means the basis-dependent Shannon–Lupanov worst-case asymptotic.

Result	Copy range	Circuit-size bound	Comparison
Uhlig [Uhl92, Weg87, Kre23]	$\log t = o(n/\log n)$	one-copy leading term	optimal order and leading constant in a smaller copy range
Paul, Lemma 2 [Pau76]	arbitrary t	$O(\max\{n2^n, nt\log^2 t\})$	$O(n2^n)$ throughout every fixed exponential range below one
Holmgren–Rothblum [HR24]	arbitrary t	$O(2^n + tn^3)$ after hardwiring	$O(2^n)$ throughout every fixed exponential range below one
Albrecht, Theorem 1 [Alb89]	$t \le 2^n$	$O(2^n n^2)$	the full exponential range, with polynomial overhead
Finite-geometry batch codes [IKOS04, PV19]	code-dependent batch size	disjoint local recovery; no bound on $\mathrm{C}(f^{\times t})$	coding antecedents for affine-geometric recovery
This paper	$t \le 2^{\gamma n}$ for fixed $\gamma < 1$	$O_\gamma(2^n/n)$	optimal order; limiting coefficient at most $1/(1-\gamma)$

Paul's construction treats the truth table as an ordered database: sort the input addresses, sweep through the hardwired table, and restore the answers to their original order. Our circuit instead sorts the records needed to select and use local recovery sets. For fixed $\gamma < 1$ and $t \le 2^{\gamma n}$, Paul's second term is $o(n2^n)$, so his displayed bound is $O(n2^n)$. The theorem proved here improves that bound by a factor of n^2 at the level of polynomial prefactors. Paul's g^k is the same k-fold product task considered here; his circuits permit arbitrary binary Boolean gates, which changes our fixed basis cost by at most a constant factor.

Holmgren and Rothblum prove that Q indexed bits can be selected from a live N-bit database by a bounded-fan-in Boolean circuit of size $O(N + Q\log^3 N)$ and depth $O(\log(N+Q))$ [HR24, Main theorem]. In our nonuniform model, hardwiring the database to the 2^n-bit truth table of f and using the t live n-bit input blocks as indices gives $O(2^n + tn^3)$, as recorded in the table. Hence the upper bound in Theorem 1.1 improves the bound obtained by this generic multiselection reduction by a factor of n throughout every fixed exponential range below one. Their result does not choose disjoint affine recovery sets, provide the sparse-to-dense scatter, or establish the bounded-congestion recursion.

Albrecht studies the same simultaneous-realization problem over a complete binary basis [Alb89, Theorem 1, pp. 53–54]. His construction handles every $t \le 2^n$ with size $O(2^n n^2)$ and depth $O(n\log n)$. Since the one-copy Shannon size is $\Theta(2^n/n)$, this is an $O(n^3)$ multiplicative overhead, as Albrecht also notes. Thus his theorem covers the endpoint copy range but does not give a constant-factor plateau at the one-copy scale.

Hiltgen and Paterson use "mass production" for PI_k-set circuits underlying monotone slice functions [HP95]. Their result gives sharing in that restricted set-circuit setting and leads to an optimal circuit for the Nečiporuk slice; it does not give a universal bound for $f^{\times t}$ in the unrestricted De Morgan circuit model considered here.

The coding interpretation begins with Uhlig's two-copy construction, which is the length-(2^p+1) single-parity-check code viewed as a two-request disjoint recovery code. Systematic multivariate polynomial evaluation, recovery on affine lines, and disjoint recovery sets are established batch-code tools, especially in Section 3.4 of Ishai–Kushilevitz–Ostrovsky–Sahai [IKOS04]. Their construction

chooses random affine lines and deletes intersections while preserving enough points for interpolation. Their Remark 3.11 also observes that the randomized decoding algorithms in that and the following sections can be derandomized using limited independence. In their terminology, the repeated-request analogue relevant here is a primitive multiset batch code. Our local gadget uses the elementary tensor-product evaluation subspace, with a separate degree bound in each variable, whereas their construction uses total-degree Reed–Muller evaluation.

That derandomization does not by itself give the circuit bound proved here. The requested prefixes are input wires of the mass-production circuit, so a schedule valid for one fixed request multiset cannot be chosen once and hardwired: the circuit must map every possible multiset of target addresses to valid recovery records, and all gates performing that map count toward $\mathrm{C}(f^{\times t})$. A generic polynomial-time or polynomial-size batch decoder would give only an unspecified polynomial bound in the exponentially large code length and batch size. Such a bound need not fit below $O(2^n/n)$ and therefore cannot justify our conclusion without finer analysis. The results cited above do not state the required fine-grained Boolean-circuit bound on this target-to-record map.

The quantitative distinction is explicit in our scheduler-sensitive composition theorem, Proposition 6.1. For a group of g requests, the explicit greedy scheduler has size $\widetilde{O}_\ell(g^2q)$. Dividing t requests among C groups changes the total scheduling cost to $\widetilde{O}_\ell(t^2q/C + Cq)$ and limits every resource to C suffixes. The bounded-congestion composition then pays for those suffixes recursively. This cost drives the equal-block induction. The nonuniform scheduler of Theorem 5.5 improves it to $\widetilde{O}_\ell(gq)$ under a constant-factor strengthening of the geometric slack condition. It permits $C = 1$ throughout every fixed exponential range and yields a direct proof.

Finite-geometry constructions of primitive batch codes likewise give explicit families of mutually disjoint recovery sets [PV19]. Later lifted-polynomial constructions also use line-based recovery to obtain batch-code guarantees [HPPV20]. Private-information-retrieval codes of Fazeli, Vardy, and Yaakobi form a related family in which each information symbol has many disjoint recovery options [FVY15]. These are coding guarantees, not gate-counted circuits for the input-dependent schedule. The self-contained proof of Theorem 1.1 uses only the elementary product-code and affine-line facts proved in Section 4.

We claim no novelty for evaluation coding, lifted high-rate codes, affine-line recovery, disjoint recovery as a combinatorial ingredient, or generic sorting and multiselection primitives. The contribution is the gate-counted composition yielding exponential-range mass production, together with the nonuniform menu scheduler and the leading-coefficient refinement. The menu argument counts every possible occupied-line state, and its implementation shares the occupied-point list across candidates. These two steps give the fine-grained bound needed by the direct proof.

11 Implications and open problems

Every fixed $\gamma < 1$ is achievable at the optimal worst-case order, and recursion can be removed in the nonuniform model. The remaining distinctions concern uniform construction, the optimal coefficient, and rates approaching one with the input length.

Can the menus be constructed efficiently? Lemma 5.4 proves that short menus exist simultaneously for all inputs. Exhaustive search is a finite construction, but no efficient uniform procedure for finding such menus is supplied.

What is the optimal leading coefficient? The normalized worst-case size has lower limiting coefficient at least 1 by one-copy counting, and Theorem 1.2 gives an upper limiting coefficient $1/(1-\gamma)$. Equality with this upper bound is not claimed. In particular, preserving coefficient 1 at a positive fixed exponential copy rate remains unresolved here.

Where does the plateau end? The theorem reaches $2^{(1-\varepsilon)n}$ copies for every fixed $\varepsilon > 0$, but does not treat a rate that approaches one with n. Some degradation in that regime is forced. Suppose $n \geq 2$ and f depends on all n of its variables. Restrict a circuit for $f^{\times t}$ to the union of its output cones, and write s for the remaining gate count. Every one of the tn inputs remains. Each connected component of the underlying undirected graph contains an output, so there are at most t components. The graph therefore has at least $tn + s - t$ edges, while fan-in at most two permits at most $2s$ edges. Hence

$$\mathrm{C}(f^{\times t}) \geq t(n-1).$$

A bound of the form $A\,2^n/n$ therefore cannot hold once $t > A\,2^n/(n(n-1))$; in particular, a uniform $O(2^n/n)$ plateau must fail for $t = \omega(2^n/n^2)$. The theorem still leaves the detailed mass-production curve throughout the near-endpoint regime $t = 2^{n-o(n)}$ unresolved. The high-rate and negligible-overhead estimates above keep ℓ, δ, and γ fixed. Substituting parameters that vary with n without new uniform estimates would not resolve this near-endpoint question.

A Circuit implementation details

This appendix records the gate-level details behind the scheduler and routing lemmas. The main proof uses only their statements and costs.

A.1 Projective-direction indexing

Proof of Lemma 5.1. Normalize a nonzero vector so that its first nonzero coordinate is one. The normalized directions whose first nonzero coordinate is in position $h \in \{1, \dots, \ell\}$ have the form

$$(0, \dots, 0, 1, w_{h+1}, \dots, w_\ell)$$

and form a block of size $q^{\ell-h}$. Let

$$B_h = \sum_{r=1}^{h-1} q^{\ell-r}, \qquad \mathrm{val}_q(w_{h+1}, \dots, w_\ell) = \sum_{r=h+1}^{\ell} \mathrm{int}(w_r) q^{\ell-r},$$

where $\mathrm{int}(w_r) \in [q]_0$ is the canonical binary representation. The rank is

$$B_h + \mathrm{val}_q(w_{h+1}, \dots, w_\ell).$$

The blocks are consecutive and their total size is D_q.

To rank, find the first nonzero coordinate, invert it, normalize the vector, and concatenate the b-bit tail coordinates. To unrank, compare the rank with the ℓ hardwired block intervals, subtract the appropriate B_h, and split the remainder into base-q digits. Since $q = 2^b$, the last step is bit slicing. The field operations and comparisons from the circuit conventions give size $\mathrm{poly}_\ell(b)$ in both directions. □

A.2 The least-missing-direction circuit

Proof of Lemma 5.2. At step j, fewer than D_q directions are forbidden, so a valid direction exists. This proves the combinatorial statement.

For the circuit bound, represent a projective direction by normalizing its first nonzero coordinate to one and use the ranking from Lemma 5.1. At stage j, for each $y \in U$, output the sentinel rank D_q when $y = u_j$ and otherwise output the projective rank of $y - u_j$. At the gate level, first multiplex a fixed nonzero vector in place of $y - u_j$ when $y = u_j$, rank that always-nonzero vector, and then overwrite the result by D_q on the equality branch. Use $\lceil \log_2(D_q + 1) \rceil$ bits per rank. There are $(j-1)(q-1) = O(jq)$ such entries.

Sort these ranks as $r_1 \le \cdots \le r_s$. The least missing rank can be found without a dense D_q-bit table. Rank zero is a candidate if $s = 0$ or $r_1 > 0$; after position i, rank $r_i + 1$ is a candidate if

$$r_i < D_q - 1 \quad \text{and} \quad (i = s \text{ or } r_{i+1} > r_i + 1).$$

These tests are applied to the full sorted list, including duplicates and sentinels. Only the last occurrence of a non-sentinel rank can generate its successor, and a sentinel cannot generate a candidate.

A left-to-right prefix-OR circuit identifies the first candidate in this order. Multiplexers select its rank using $O(s \log D_q)$ gates.

Since $|U \setminus \{u_j\}| < D_q$, fewer than D_q distinct non-sentinel ranks are forbidden, so a candidate exists. Sorting, comparisons, and priority selection therefore cost $\widetilde{O}_\ell(jq)$ gates. Unrank the selected direction using Lemma 5.1 and enumerate its $q-1$ non-target line points, at cost $q\,\mathrm{poly}_\ell(b)$. Unrolling the stages and summing over $j \le g$ gives $\widetilde{O}_\ell(g^2 q)$. □

A.3 Four-pass fixed-wire routing

Proof of Lemma 5.6. For scatter, create one slot record for each of the Cq^ℓ keys and combine them with the fewer than tq incidence records. Preserve a copy of every incidence record by free fan-out. Uniqueness inside a group puts at most one incidence at each key. After the first pass in the table, a slot therefore copies the preceding suffix exactly when a type-and-key equality test succeeds; otherwise it receives a fixed dummy suffix. The predecessor of the first position is defined to be a dummy record, so the boundary case uses the same guard. The second pass places the filled slots in canonical (group, y) order. Fixed wiring transposes this order to (y, group), the input order used by the resource circuits.

For gather, combine the preserved incidences with one value record per key. The third pass places that value immediately before the unique matching incidence; a guarded local multiplexer copies it into the incidence. The fourth pass puts the first $t(q-1)$ positions in the fixed request-and-line order required by the decoders. Thus absent incidences, the first array position, and all type boundaries are handled by fixed dummy data and the same guarded update, not by data-dependent wiring.

This is a constant number of sorting networks on $O(tq + Cq^\ell)$ records. Each record has width $\mathrm{poly}_\ell(n, \log q, \log t, \log C)$, so the standard primitives from the circuit conventions give the stated bound. □

AI Disclosure

This project began in an interactive conversation on ChatGPT.com that was exported to the author's computer as a LaTeX document. Later interactive sessions used Claude Fable 5 and 5.1 through Claude Code and ChatGPT 5.6 Sol through Codex. These systems materially influenced the mathematical experiments, broad literature searches, proof strategy, and internal constructions, including the scheduling and recursive-composition approach; their role went beyond prose editing. The author selected the directions to pursue, checked and revised the proofs and literature claims, and takes full responsibility for the correctness, originality, and integrity of the paper. The present revision also used Codex to develop and test the nonuniform menu scheduler and the high-rate-code refinement, and to complete their Lean formalization through the sharp real-rate theorem.

References

[Alb89] Andreas Albrecht. On simultaneous realizations of Boolean functions, with applications. In *Parcella '88: Proceedings of the Fourth International Workshop on Parallel Processing by Cellular Automata and Arrays*, LNCS 342, pages 51–56. Springer, 1989. doi:10.1007/3-540-50647-0_102.

[Bat68] Kenneth E. Batcher. Sorting networks and their applications. In *Proceedings of the 1968 Spring Joint Computer Conference*, AFIPS Conference Proceedings 32, pages 307–314, 1968. doi:10.1145/1468075.1468121.

[FVY15] Arman Fazeli, Alexander Vardy, and Eitan Yaakobi. PIR with low storage overhead: Coding instead of replication. arXiv:1505.06241, 2015. https://arxiv.org/abs/1505.06241.

[FM05] Gudmund Skovbjerg Frandsen and Peter Bro Miltersen. Reviewing bounds on the circuit size of the hardest functions. *Information Processing Letters*, 95(2):354–357, 2005. doi:10.1016/j.ipl.2005.03.009.

[GKS13] Alan Guo, Swastik Kopparty, and Madhu Sudan. New affine-invariant codes from lifting. In *Proceedings of the 4th Innovations in Theoretical Computer Science Conference*, pages 529–539. ACM, 2013. doi:10.1145/2422436.2422494. https://arxiv.org/abs/1208.5413.

[HP95] Alain P. Hiltgen and Mike S. Paterson. PI_k mass production and an optimal circuit for the Nečiporuk slice. *Computational Complexity*, 5(2):132–154, 1995. doi:10.1007/BF01268142.

[HPPV20] Lukas Holzbaur, Rina Polyanskaya, Nikita Polyanskii, and Ilya Vorobyev. Lifted Reed–Solomon codes with application to batch codes. In *2020 IEEE International Symposium on Information Theory*, pages 634–639. IEEE, 2020. doi:10.1109/ISIT44484.2020.9174402. arXiv:2001.11981.

[HR24] Justin Holmgren and Ron D. Rothblum. Linear-size Boolean circuits for multiselection. In *39th Computational Complexity Conference (CCC 2024)*, LIPIcs 300, Article 11, pages 11:1–11:20, 2024. doi:10.4230/LIPIcs.CCC.2024.11.

[IKOS04] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Batch codes and their applications. In *Proceedings of the 36th Annual ACM Symposium on Theory of Computing*, pages 262–271. ACM, 2004. doi:10.1145/1007352.1007396.

[Kre23] William Kretschmer. Quantum mass production theorems. In *18th Conference on the Theory of Quantum Computation, Communication and Cryptography*, LIPIcs 266, Article 10, pages 10:1–10:11, 2023. doi:10.4230/LIPIcs.TQC.2023.10.

[Lup58] Oleg B. Lupanov. On a method of circuit synthesis. *Izvestiya VUZ, Radiofizika*, 1(1):120–140, 1958. In Russian. https://radiophysics.unn.ru/issues/1958/1/120.

[Pau76] Wolfgang J. Paul. Realizing Boolean functions on disjoint sets of variables. *Theoretical Computer Science*, 2(3):383–396, 1976. doi:10.1016/0304-3975(76)90089-X.

[PV19] Nikita Polyanskii and Ilya Vorobyev. Constructions of batch codes via finite geometry. In *2019 IEEE International Symposium on Information Theory*, pages 360–364. IEEE, 2019. doi:10.1109/ISIT.2019.8849736. arXiv:1901.06741.

[Sha49] Claude E. Shannon. The synthesis of two-terminal switching circuits. *Bell System Technical Journal*, 28(1):59–98, 1949. doi:10.1002/j.1538-7305.1949.tb03624.x.

[Uhl74] Dietmar Uhlig. On the synthesis of self-correcting schemes from functional elements with a small number of reliable elements. *Mathematical Notes of the Academy of Sciences of the USSR*, 15(6):558–562, 1974. English translation. doi:10.1007/BF01152835.

[Uhl92] Dietmar Uhlig. Networks computing Boolean functions for multiple input values. In M. S. Paterson, editor, *Boolean Function Complexity*, London Mathematical Society Lecture Note Series 169, pages 165–173. Cambridge University Press, 1992. doi:10.1017/CBO9780511526633.013.

[Weg87] Ingo Wegener. *The Complexity of Boolean Functions*. Wiley–Teubner, 1987. https://eccc.weizmann.ac.il/static/books/The_Complexity_of_Boolean_Functions/.